

Reviewer Pack — Euler Characteristic of Gate-Conflict Independence Complex B...

Entry cdef279c90bb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 19:02:19 UTC

Euler Characteristic of Gate-Conflict Independence Complex Bounds $ACC^0[m]$ Size for MOD_q

Entry ID: cdef279c90bb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 19:02:19 UTC

1. Conjecture

Field A (mathematical branch): Topological combinatorics — reduced Euler characteristic $\tilde{\chi}(\text{Ind}(G))$ of the independence complex of a finite graph G , computed as the independence-polynomial value $I(G;-1)$ (Kozlov 2008 ‘Combinatorial Algebraic Topology’; Engström 2009 ‘Complexes of directed trees and independence complexes’; Stanley-Reisner / Alexander duality). This corner of combinatorial algebraic topology has essentially no presence in circuit complexity (arXiv ‘independence complex’ AND (‘ ACC^0 ’ OR ‘circuit complexity’ OR ‘Williams algorithmic method’) returns no direct hits; <5 adjacent monotone-circuit-matroid papers). **Field B** (complexity object): $ACC^0[m]$ circuit size for MOD_q on n inputs with q coprime to m (focus $q=3$, $m=2$), in the Williams 2011 / Murray-Williams 2018 algorithmic-method regime targeting $NEXP \not\subseteq ACC^0$. The invariant is a STRUCTURAL property of the circuit DAG (gate-conflict graph), not of the truth table, so it is shielded from the Razborov-Rudich natural-proofs barrier.

Statement:

For an $ACC^0[m]$ circuit C of depth d and size s on n inputs, let G_C be the gate-conflict graph: vertices = non-input gates; (g,h) is an edge iff the input-variable supports $\text{supp}(g), \text{supp}(h) \subseteq [n]$ satisfy $\text{supp}(g) \cap \text{supp}(h) \neq \emptyset$. Let $\tau(C) := |I(G_C;-1)|$ where $I(G;x) = \sum_{S \text{ independent in } G} x^{|S|}$. Conjecture: every depth- d $ACC^0[m]$ circuit C that ε -approximates MOD_q on n inputs (q coprime to m , $\varepsilon \leq 1/4$, $q \geq 3$) satisfies $\tau(C) \geq 2^{\lfloor n/(10 \cdot d^2) \rfloor} - 1$. A single (C, q, ε) instance with $\tau(C) < 2^{\lfloor n/(10 \cdot d^2) \rfloor} - 1$ yet $\text{agreement}(C, MOD_q) \geq (1-\varepsilon) \cdot 2^n$ refutes it.

Rationale (proposer’s reasoning):

Razborov-Smolensky shows that approximating MOD_q in $AC^0[p]$ forces large size via polynomial-method incompatibility; the independence complex of the gate-conflict graph categorifies the variable-overlap pattern that polynomial-degree bounds exploit, with $\tilde{\chi}$ measuring ‘how rigidly’ supports must intersect to express many distinct minterms. Because $\tau(C)$ depends only on the wiring DAG (not the truth table), the invariant cannot be ‘large + useful’ in the Razborov-Rudich sense, dodging natural proofs; because it is built from set intersections and Möbius sign sums (not ring operations on tt-extensions), it does not algebrize in the Aaronson-Wigderson sense. The bound $2^{\lfloor n/(10 \cdot d^2) \rfloor}$ aligns with Smolensky’s $\sqrt{n}$ degree barrier propagated through d layers.

Taxonomy category: $ACC_LB_via_WILLIAMS$ (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a5e158ae307482fd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{6, 8, 10, 12\}$, $d \in \{2, 3, 4\}$, $s \in \{8, 12, 16, 20\}$, 30 seeds each: among circuits with $\text{agreement}(C, \text{MOD}_3) \geq (3/4) \cdot 2^n$, the bound $\tau(C) \geq 2^{\{n/(10 \cdot d^2)\} - 1}$ must hold in $\geq 90\%$ of qualifying seeds; any single qualifying (C, n, d) violating it falsifies the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.82	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - independence complex Euler characteristic circuit complexity lower bound - ACC0 MOD_q lower bound topological method algebraic combinatorics - gate conflict graph independence polynomial circuit depth size

Top relevant hits considered: - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1606.05050v1>] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [<http://arxiv.org/abs/cs/9906008v2>] A Lower Bound on the Average-Case Complexity of Shellsort - [<http://arxiv.org/abs/2304.04810v2>] Chain algebras of finite distributive lattices - [<http://arxiv.org/abs/1110.6876v4>] Lower bounds for topological complexity - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [<http://arxiv.org/abs/1606.00596v3>] Randomized Polynomial Time Identity Testing for Noncommutative Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_random_circuit(n, d, s):
    gates = []
    for _ in range(s):
```

```

    if len(gates) < 2:
        gate_type = random.choice(['AND', 'OR', 'NOT', 'XOR'])
    else:
        gate_type = random.choice(['AND', 'OR', 'NOT', 'XOR', 'INPUT'])
    if gate_type == 'INPUT':
        inputs = [random.randint(0, n-1)]
    else:
        inputs = random.sample(range(len(gates)), min(2, len(gates)))
    gates.append((gate_type, inputs))
return gates

def compute_truth_table(circuit, n):
    truth_table = {}
    for inputs in itertools.product([0, 1], repeat=n):
        input_values = list(inputs)
        for gate in circuit:
            gate_type, gate_inputs = gate
            if gate_type == 'INPUT':
                output = input_values[gate_inputs[0]]
            elif gate_type == 'AND':
                output = input_values[gate_inputs[0]] & input_values[gate_inputs[1]]
            elif gate_type == 'OR':
                output = input_values[gate_inputs[0]] | input_values[gate_inputs[1]]
            elif gate_type == 'NOT':
                output = 1 - input_values[gate_inputs[0]]
            elif gate_type == 'XOR':
                output = input_values[gate_inputs[0]] ^ input_values[gate_inputs[1]]
            input_values.append(output)
        truth_table[inputs] = input_values[-1]
    return truth_table

def compute_supports(circuit, n):
    supports = []
    for gate in circuit:
        gate_type, gate_inputs = gate
        if gate_type == 'INPUT':
            support = [gate_inputs[0]]
        else:
            support = []
            for i in gate_inputs:
                support.extend(supports[i])
            support = list(set(support))
        supports.append(support)
    return supports

def build_gate_conflict_graph(supports):
    graph = defaultdict(set)
    for i in range(len(supports)):
        for j in range(i+1, len(supports)):
            if set(supports[i]) & set(supports[j]):
                graph[i].add(j)
                graph[j].add(i)
    return graph

def is_independent_set(graph, S):
    for i in S:

```

```

        for j in S:
            if i != j and j in graph[i]:
                return False
        return True

def compute_tau(graph):
    tau = 0
    for k in range(1, len(graph)+1):
        for S in itertools.combinations(graph.keys(), k):
            if is_independent_set(graph, S):
                tau += (-1)**k
    return tau

def compute_agreement(truth_table, mod_q):
    agreement = 0
    for inputs in truth_table:
        x = sum(inputs) % mod_q
        if truth_table[inputs] == (x != 0):
            agreement += 1
    return agreement

def run_trial(seed):
    random.seed(seed)
    n = random.choice([6, 8, 10, 12])
    d = random.choice([2, 3, 4])
    s = random.choice([8, 12, 16, 20])

    circuit = generate_random_circuit(n, d, s)
    truth_table = compute_truth_table(circuit, n)
    supports = compute_supports(circuit, n)
    graph = build_gate_conflict_graph(supports)
    tau = compute_tau(graph)

    mod_q = random.choice([3, 5])
    agreement = compute_agreement(truth_table, mod_q)
    threshold = (3/4) * (2**n)

    predicted_bound = 2**(n / (10 * d**2)) - 1
    conjecture_holds = tau >= predicted_bound if agreement >= threshold else True

    counterexample = ""
    if not conjecture_holds and agreement >= threshold:
        counterexample = f"tau={tau} < {predicted_bound} for n={n}, d={d}, s={s}, mod_q={mod_q}"

    return {
        "metric_name": "tau",
        "metric_value": tau,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [random.randint(1, 1000)]

    metric_values = []

```

```

conjecture_holds_counts = 0
total_instances = 0

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    metric_values.append(result["metric_value"])
    if result["conjecture_holds"]:
        conjecture_holds_counts += 1
    total_instances += result["instances_tested"]

mean_metric = sum(metric_values) / len(metric_values)
std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
support_fraction = conjecture_holds_counts / len(seeds)

if support_fraction >= 0.9:
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
else:
    for seed in seeds:
        result = run_trial(seed)
        if not result["conjecture_holds"] and result["counterexample"]:
            print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_fail={seed}")
            break
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbad24de.py", line 135, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbad24de.py", line 103, in run_trial
    truth_table = compute_truth_table(circuit, n)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cbad24de.py", line 45, in compute_truth
    output = input_values[gate_inputs[0]] ^ input_values[gate_inputs[1]]
              ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` before producing any data, so the pre-registered support condition could not be evaluated and no qualifying instances were assessed. | next: Fix the circuit-evaluation bug (guard `gate_inputs` indexing against gates with fewer than 2 inputs, or generate circuits whose gate fan-in matches the evaluator) and rerun the trial sweep.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	223797
2	propose	claude_max	opus	0	0	314389
3	preregistration	claude_max	opus	0	0	6335
4	novelty	claude_max	opus	0	0	3225
5	novelty	claude_max	opus	0	0	10921
6	test_gen	mistral	codestral-latest	0	0	329779
7	test_gen	mistral	codestral-latest	0	0	315600
8	test_gen	mistral	codestral-latest	0	0	311707
9	test_gen	mistral	codestral-latest	0	0	310966
10	judge	claude_max	opus	0	0	9600

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1836319 ms total latency. Provider mix: {'claude_max': 6, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/cdef279c90bb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cdef279c90bb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cdef279c90bb.tar.gz` (if generated)